

Artificial Intelligence (CS24307) — Full Course Master Book

Complete End-to-End B.Tech CSE 5th Semester Study Book & PYQ Bank

EXECUTIVE MASTER STUDY GUIDE & ALGORITHMIC PROBLEM BANK

Artificial Intelligence (CS24307)

Birla Institute of Technology, Mesra | B.Tech CSE 5th Semester (NEP 2024–25 Scheme)

Complete Course Structure & Algorithmic Matrix

Module	Core Syllabus Scope	Key Algorithms & Formulations
Module I	Intelligent Agents & Problem Solving	PEAS Description, Environment Types, State Space Formulation, Uninformed Search (BFS, DFS, DLS, IDDFS, UCS)
Module II	Informed Search & Game AI	Greedy Best-First, A* Search (Admissibility & Consistency Proofs), IDA*, Hill Climbing, Simulated Annealing, Minimax, Alpha-Beta Pruning, CSP
Module III	Knowledge & Logic Resolution	Propositional Logic (PL), First-Order Logic (FOL), CNF Conversion, Forward & Backward Chaining, Resolution Refutation
Module IV	Planning & Probabilistic Reasoning	STRIPS, PDDL Action Schemas, Planning Graphs, Bayesian Belief Networks (BBN), Conditional Independence, d-separation
Module V	Learning & Neural Networks	Inductive Learning, Decision Tree (ID3 Information Gain), Perceptrons, Multi-Layer Perceptrons (MLP), Backpropagation

Exam Preparation & High-Yield Strategy

This publication-grade master book consolidates all 5 modules with formal search completeness & optimality proofs, KaTeX-rendered logic formulations, game tree pruning traces, and model answers to BIT Mesra end-semester examination questions.

Module I: Preliminaries & Intelligent Agents — Complete 7-Topic Syllabus Tracker

Topic 1: What is AI? (4 Classical Approaches & Rational Agent Paradigm)

Topic 3: Intelligent Agents (Sense-Think-Act Loop & PEAS Framework)

Topic 5: Nature of Environments (O-D-E-D-C-N Taxonomy)

Topic 7: Real-World Applications of AI (Healthcare, Robotics, Vision)

Topic 2: Evolution of AI (1950 Turing Test to 2020s Generative AI)

Topic 4: Concept of Rationality (Rationality vs. Omniscience)

Topic 6: Structure of Agents (5 Architectural Agent Types)

Topic 1 & 2: Definitions, Approaches & Historical Evolution of AI

Artificial Intelligence is the branch of computer science dedicated to designing computational systems that synthesize capabilities traditionally associated with human intelligence: reasoning, perception, learning, planning, and goal-directed decision making.

Approach	Philosophical / Methodological Basis	Standard Evaluation Benchmark
1. Thinking Humanly	Cognitive science modeling of the inner workings of human thought processes.	GPS (General Problem Solver), Cognitive Neuroimaging.
2. Acting Humanly	Empirical behavioral replication of human performance.	Turing Test (1950) by Alan Turing.
3. Thinking Rationally	Formal logic and laws of thought (Syllogisms, First-Order Logic).	Deductive Inference Engines and Proof Systems.
4. Acting Rationally	Rational Agent Approach (Modern Standard): Maximizing expected performance given available information.	Goal achievement and expected utility maximization.

🧠 Memory Hook: Evolution Timeline (T-D-E-ML-DL-GenAI)

1950: Turing Test → **1956:** Dartmouth Conference (John McCarthy coining "AI") → **1980s:** Expert Systems (R1/XCON) → **1997:** Deep Blue defeats Kasparov → **2011:** IBM Watson wins Jeopardy → **2016:** AlphaGo defeats Lee Sedol → **2020s:** Large Multimodal Transformers & Generative AI.

Topic 3 & 4: Intelligent Agents, PEAS & The Concept of Rationality

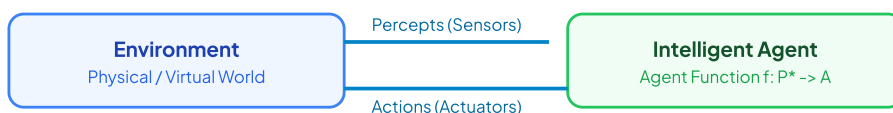


Figure 1.1: The Core Sense-Think-Act Intelligent Agent Interaction Loop

Rationality $\neq$ Omniscience

An **omniscient agent** knows the *actual outcome* of its actions in advance and acts with superhuman clairvoyance (impossible in reality). A **rational agent** chooses the action that maximizes its *expected performance measure*, given its accumulated percept history sequence P^* and built-in knowledge base!

PEAS Specification Framework (Autonomous Taxi Example):

- P — Performance Measure:** Safety, passenger comfort, speed, legal compliance, fuel economy, profit.
- E — Environment:** City streets, highways, pedestrians, weather conditions, other vehicular traffic.
- A — Actuators:** Steering wheel, accelerator, brake pedal, horn, turn signals, display screen.
- S — Sensors:** Video cameras, LiDAR, radar, ultrasonic sonar, GPS, speedometer, odometer, engine sensors.

Topic 5: Nature of Environments (O-D-E-D-C-N Taxonomy)

Environment Dimension	Core Conceptual Distinction	Representative Example
1. Fully vs. Partially Observable	Sensors detect the complete state of the world vs. state is obscured by noise or missing sensors.	Chess (Fully) vs. Poker / Driving (Partially)
2. Deterministic vs. Stochastic	Next state determined 100% by current state + action vs. randomness and uncertainty exist.	Crossword (Deterministic) vs. Weather / Taxi (Stochastic)
3. Episodic vs. Sequential	Each action episode is independent vs. current action alters future states and rewards.	Image Defect Sorting (Episodic) vs. Chess (Sequential)
4. Static vs. Dynamic	Environment remains unchanged while agent deliberates vs. changes continuously in real time.	Sudoku (Static) vs. Autonomous Driving (Dynamic)
5. Discrete vs. Continuous	Countable discrete states and actions vs. continuously varying physical quantities.	Tic-Tac-Toe (Discrete) vs. Robot Arm Control (Continuous)
6. Single-Agent vs. Multi-Agent	Agent operates alone vs. multiple agents interact cooperatively or competitively.	Crossword Puzzle (Single) vs. Chess / Stock Market (Multi)

Topic 6: The 5 Architectural Agent Types

Agent Type	Decision Mechanism & Internal State	Key Limitation / Advantage
1. Simple Reflex	Direct Condition-Action Rules ('IF condition THEN action') based strictly on current percept.	Zero memory; fails completely in partially observable environments.
2. Model-Based Reflex	Maintains internal state reflecting "how the world evolves" and "how actions affect the world".	Overcomes partial observability by tracking hidden state history.
3. Goal-Based	Combines internal state with explicit Goal information using Search & Planning algorithms.	Flexible: goals can be altered dynamically without rewriting rules.
4. Utility-Based	Evaluates state desirability using an explicit real-valued Utility Function $U(s)$.	Handles conflicting goals and probabilistic risk-reward tradeoffs.
5. Learning Agent	Decoupled into 4 Components : (1) Learning Element, (2) Performance Element, (3) Critic (evaluator), (4) Problem Generator (exploration).	Autonomously improves performance over time from raw environmental feedback.



M1 Active Recall & Exam Questions

Q1. Define an Intelligent Agent. Explain the 4 components of a Learning Agent with a neat architecture diagram. (10 Marks)

An **Intelligent Agent** is an autonomous entity that perceives its environment through sensors and takes actions using actuators to achieve its goals ($f : P^* \rightarrow A$).

The 4 Components of a Learning Agent:

1. *Learning Element*: Responsible for making improvements by modifying the agent's internal behavior rules.
2. *Performance Element*: Responsible for selecting external actions based on percepts (the operational agent).
3. *Critic*: Evaluates the agent's performance against an external performance standard and provides feedback to the learning element.
4. *Problem Generator*: Suggests experimental exploratory actions that lead to new, informative experiences rather than suboptimal repetition.

Module II: Problem Solving by Search Agents — Complete 6-Topic Syllabus Tracker

Topic 8: Search-Based Problem Solving (5-Tuple Formal Problem Formulation)

Topic 11: Heuristic Search (Greedy Best-First, A* Search & Admissibility)

Topic 13: Search in Complex Environments (Belief States & Contingencies)

Topic 9 & 10: State-Space Search (BFS, DFS, UCS, DLS, IDDFS & Graph Search)

Topic 12: Local Search (Hill Climbing, Simulated Annealing & Genetic Algo)

Topic 14: Adversarial Game-Tree Search (Minimax Algorithm & Alpha-Beta Pruning)

Topic 8: Search Problem Formulation (The 5-Tuple)

Formal Problem Formulation 5-Tuple: $\langle S_0, A(s), \text{Result}(s, a), \text{GoalTest}(s), c(s, a, s') \rangle$

1. **Initial State** (S_0): The starting state the agent begins in (e.g., $\text{In}(\text{Arad})$).
2. **Actions** ($A(s)$): Set of legal executable actions available in state s .
3. **Transition Model** ($\text{Result}(s, a)$): Returns the successor state after taking action a in state s .
4. **Goal Test** ($\text{GoalTest}(s)$): Boolean function determining if state s is a target goal.
5. **Path Cost Function** ($c(s, a, s')$): Step cost $g(n)$ of taking action a to reach s' . Total path cost = $\sum c$.

Topic 9 & 10: Uninformed (Blind) Search Strategies

Algorithm	Frontier Data Structure	Time Complexity	Space Complexity	Complete?	Optimal?
1. BFS	FIFO Queue	$O(b^{d+1})$	$O(b^{d+1})$ (High)	Yes	Yes (if equal step costs)
2. DFS	LIFO Stack / Recursion	$O(b^m)$	$O(b \cdot m)$ (Low)	No (in infinite trees)	No
3. UCS	Priority Queue by $g(n)$	$O(b^{1+\lceil C^*/\epsilon \rceil})$	$O(b^{1+\lceil C^*/\epsilon \rceil})$	Yes	Yes (for step costs $\geq \epsilon > 0$)
4. IDDFS	Iterative LIFO Stack	$O(b^d)$	$O(b \cdot d)$ (Low)	Yes	Yes (if equal step costs)

Topic 11: Informed Heuristic Search & The A* Algorithm

1. Evaluation Functions: - **Greedy Best-First Search:** $f(n) = h(n)$ (Fast, but non-optimal; can get trapped in loops). - **A* Search:** $f(n) = g(n) + h(n)$ where $g(n)$ is the exact cost from start to n , and $h(n)$ is estimated cost to goal.

2. Admissibility and Consistency Conditions for A* Optimality: - **Admissible Heuristic:** $h(n) \leq h^*(n)$ for all n (never overestimates the true remaining cost). Guarantees A* is optimal in Tree Search! - **Consistent (Monotonic) Heuristic:** $h(n) \leq c(n, a, n') + h(n')$ (satisfies the triangle inequality). Guarantees A* is optimal in Graph Search with zero node re-expansions!

Topic 12: Local Search & Optimization Algorithms

Local Search Algorithm	Operating Strategy & State Space Navigation	Key Pitfall & Solution
1. Hill Climbing	Greedy local search that continuously moves in the direction of increasing elevation (value).	Gets trapped on Local Maxima, Plateaus, and Ridges . Solution: Random-restart hill climbing.
2. Simulated Annealing	Escapes local maxima by allowing downhill moves with probability $P = e^{\frac{\Delta E}{T}}$. Temperature T decreases according to a cooling schedule.	Guaranteed asymptotic convergence to global optimum as $T \rightarrow 0$ if cooled sufficiently slowly.
3. Genetic Algorithms	Population-based search simulating natural selection: Selection (fitness proportionate) $\rightarrow$ Crossover $\rightarrow$ Mutation .	Excellent for massive, complex, multi-modal search landscapes (e.g., TSP, circuit design).

Topic 14: Adversarial Game-Tree Search (Minimax & Alpha-Beta Pruning)

1. Minimax Value Recursive Definition:

$$\text{Minimax}(s) = \begin{cases} \text{Utility}(s) & \text{if Terminal}(s) \\ \max_{a \in A(s)} \text{Minimax}(\text{Result}(s, a)) & \text{if Player}(s) = \text{MAX} \\ \min_{a \in A(s)} \text{Minimax}(\text{Result}(s, a)) & \text{if Player}(s) = \text{MIN} \end{cases}$$

Alpha-Beta ($\alpha - \beta$) Pruning Invariant Rule

Maintain two bounds during recursive DFS traversal:

- α = the highest value choice found so far along the path for **MAX** (initialized to $-\infty$).
- β = the lowest value choice found so far along the path for **MIN** (initialized to $+\infty$).

Pruning Condition: Whenever $\alpha \geq \beta$, prune the remaining subtree below the current node immediately! (Reduces effective branching factor from b to $\sqrt{b}$, doubling search depth!).

M2 Active Recall & Exam Questions

Q1. Prove that A* Search is optimal when using an Admissible Heuristic in Tree Search. (8 Marks)

Let G_2 be a suboptimal goal with cost $g(G_2) > C^*$, and G be an optimal goal with $g(G) = C^*$.

Suppose G_2 is in the frontier and is about to be selected. For any unexpanded node n on the optimal path to G :

- Since h is admissible: $f(n) = g(n) + h(n) \leq C^*$.
- For the suboptimal goal: $f(G_2) = g(G_2) + h(G_2) = g(G_2) > C^*$.
- Therefore, $f(n) \leq C^* < f(G_2) \implies f(n) < f(G_2)$.

Hence, A* will always expand node n on the optimal path before ever selecting G_2 , guaranteeing that G_2 can never be chosen before an optimal path is found!

Module III: Knowledge Representation & Logical Reasoning — Complete 9-Topic Tracker

Topic 15: Knowledge-Based Agents (KB, Inference Engine & TELL/ASK)

Topic 17: Propositional to Predicate Logic Transition

Topic 19: First-Order Predicate Logic (Constants, Predicates & Quantifiers)

Topic 21: Forward Chaining (Data-Driven Deductive Reasoning)

Topic 23: The Resolution Inference Rule (Refutation Proofs & CNF)

Topic 16: Propositional Logic (Syntax, Semantics, Truth Tables & Equivalences)

Topic 18: Propositional Logic-Based Agents (Wumpus World)

Topic 20: Knowledge Representation in FOL (English to Logic Translation)

Topic 22: Backward Chaining (Goal-Driven Hypothesis Verification)

Topic 15: Knowledge-Based Agents (KB Architecture)

A **Knowledge-Based Agent (KBA)** uses a formal, explicit internal representation of knowledge (the Knowledge Base KB) and an Inference Engine to deduce new facts and make decisions:

TELL(KB , sentence) $\leftarrow$ Adds new percept facts/rules to KB
ASK(KB , query) $\leftarrow$ Deduces whether $KB \models \text{query}$ (Entailment)

Topic 16 & 17: Propositional Logic vs. First-Order Logic (FOL)

Formal Logic System	Ontological Commitment	Epistemological Commitment
Propositional Logic	Facts that are either True or False (Boolean propositions P, Q). Treats atomic sentences as indivisible black boxes.	Belief states: {True, False, Unknown}.
First-Order Logic (FOL)	Objects (Constants), Relations (Predicates), Functions , and Quantifiers ($\forall, \exists$).	Belief states: {True, False, Unknown}. Expressive power to model complex domains.

Key Propositional Equivalence Laws: - Implication Elimination: $P \implies Q \equiv \neg P \vee Q$ - Biconditional Elimination: $P \iff Q \equiv (P \implies Q) \wedge (Q \implies P)$ - De Morgan's Laws: $\neg(P \wedge Q) \equiv \neg P \vee \neg Q$, $\neg(P \vee Q) \equiv \neg P \wedge \neg Q$

Topic 19 & 20: Knowledge Representation in First-Order Logic (FOL)

Classic English $\rightarrow$ First-Order Logic Translation Suite

Natural English Statement	First-Order Predicate Logic Formulation
"Every student is intelligent."	$\forall x (\text{Student}(x) \implies \text{Intelligent}(x))$
"Some students like AI."	$\exists x (\text{Student}(x) \wedge \text{Likes}(x, \text{AI}))$
"No person can live on the sun."	$\neg \exists x (\text{Person}(x) \wedge \text{LivesOn}(x, \text{Sun})) \equiv \forall x (\text{Person}(x) \implies \neg \text{LivesOn}(x, \text{Sun}))$
"Everyone has a mother."	$\forall x \exists y \text{MotherOf}(y, x)$

Topic 21 & 22: Forward Chaining vs. Backward Chaining

Parameter	Forward Chaining (Data-Driven)	Backward Chaining (Goal-Driven)
Starting Point	Starts from known initial atomic facts in the KB.	Starts from the Goal query hypothesis Q .
Execution Direction	Applies Modus Ponens forward: $P, P \implies Q \implies \text{infer } Q$. Adds new facts until goal is reached.	Works backward: finds rules that have Q in the conclusion, and recursively proves their premises.
Suitability	Ideal for monitoring, diagnosis, and reactive synthesis systems.	Ideal for targeted automated theorem proving, diagnosis, and query answering (Prolog).

Topic 23: The Resolution Refutation Algorithm

The Resolution Inference Rule:

$$\frac{\ell_1 \vee \dots \vee \ell_i \vee \dots \vee \ell_k, \quad m_1 \vee \dots \vee \neg \ell_i \vee \dots \vee m_n}{\ell_1 \vee \dots \vee \ell_{i-1} \vee \ell_{i+1} \vee \dots \vee \ell_k \vee m_1 \vee \dots \vee m_{j-1} \vee m_{j+1} \vee \dots \vee m_n}$$

Algorithm: Proof by Resolution Refutation ($KB \models \alpha$)

1. Convert all sentences in KB into **Conjunctive Normal Form (CNF)** (Eliminate $\implies$, push $\neg$ inward, standardize variables, Skolemize $\exists$, drop $\forall$, distribute $\vee$ over $\wedge$).
2. **Negate the Query:** Add $\neg \alpha$ to the CNF knowledge base.
3. Repeatedly apply the Resolution Rule to pairs of complementary literals (ℓ and $\neg \ell$) to generate new resolvents.
4. If resolution derives the **Empty Clause** ($\square$), a formal logical contradiction is achieved $\implies$ Original query α is mathematically proven!

M3 Active Recall & Exam Questions

Q1. Given $KB = \{P \implies Q, Q \implies R, P\}$, prove R using Resolution Refutation. (8 Marks)

Step 1: Convert KB to CNF Clauses:

- (1) $P \implies Q \equiv \neg P \vee Q$
- (2) $Q \implies R \equiv \neg Q \vee R$
- (3) P

Step 2: Add Negation of Goal R :

- (4) $\neg R$

Step 3: Resolution Steps:

- (5) Resolve (1) and (3) on literal $P \implies Q$
 - (6) Resolve (2) and (5) on literal $Q \implies R$
 - (7) Resolve (4) and (6) on literal $R \implies \square$ (Empty Clause / Contradiction).
- $\therefore R$ is proven true by refutation!

Module IV: Planning & Probabilistic Reasoning — Complete 7-Topic Syllabus Tracker

Topic 24: Planning in AI (Goal-Directed Action Sequences)

Topic 25: Components of Planning Problems (STRIPS Operators: PRE, ADD, DEL)

Topic 26: Types of Planning (State-Space, Partial-Order & Hierarchical HTN)

Topic 27: Goal Stack Planning (Subgoal Interactions & Sussman Anomaly)

Topic 28: Reasoning Under Uncertainty (Axioms of Probability)

Topic 29: Bayesian Inference (Prior, Likelihood, Posterior & Evidence)

Topic 30: Probabilistic Reasoning & Bayesian Belief Networks (DAGs)

Topic 24 & 25: Planning Foundations & STRIPS Representation

STRIPS Action Operator Representation (Fikes & Nilsson, 1971)

An action A is represented as a triple:

- Action Name:** e.g., $\text{Move}(b, x, y)$ (Move block b from surface x to surface y).
- Preconditions (PRE):** Set of positive logical literals that must be true for action to execute ($\text{On}(b, x) \wedge \text{Clear}(b) \wedge \text{Clear}(y)$).
- Add-List (ADD):** Set of literals made true by executing the action ($\text{On}(b, y) \wedge \text{Clear}(x)$).
- Delete-List (DEL):** Set of literals made false by executing the action ($\text{On}(b, x) \wedge \text{Clear}(y)$).

Topic 26 & 27: Goal Stack Planning & Types of Planning

Planning Paradigm	Operating Mechanism	Key Limitation / Advantage
1. Goal Stack Planning	Maintains a LIFO stack of subgoals and STRIPS operators. Solves subgoals sequentially one by one.	Vulnerable to Subgoal Interleaving (Sussman Anomaly) : achieving one subgoal may inadvertently undo a previously satisfied subgoal.
2. Partial-Order Planning (POP)	Works in plan space; adds actions and orders them only when strictly necessary (Least-Commitment principle).	Eliminates unnecessary search commitments; handles interleaved subgoals gracefully.
3. Hierarchical Task Network (HTN)	Recursively decomposes high-level abstract tasks into executable primitive action sequences.	Scales to industrial-scale engineering and manufacturing planning problems.

Topic 28 & 29: Reasoning Under Uncertainty & Bayesian Inference

1. Fundamental Axioms of Probability: - Range: $0 \leq P(A) \leq 1$, $P(\text{True}) = 1$, $P(\text{False}) = 0$ - Disjunction: $P(A \vee B) = P(A) + P(B) - P(A \wedge B)$ - Conditional Probability: $P(A | B) = \frac{P(A \wedge B)}{P(B)} \implies P(A \wedge B) = P(A | B)P(B)$

2. Bayes' Rule Formulation (Bayes, 1763):

$$P(\text{Hypothesis} | \text{Evidence}) = \frac{P(\text{Evidence} | \text{Hypothesis}) \cdot P(\text{Hypothesis})}{P(\text{Evidence})}$$

$$\text{Posterior} = \frac{\text{Likelihood} \times \text{Prior}}{\text{Evidence}}$$

Step-by-Step Bayesian Medical Diagnosis Problem

A disease affects 1% of the population ($P(D) = 0.01$). A test has 95% true positive rate ($P(T | D) = 0.95$) and 5% false positive rate ($P(T | \neg D) = 0.05$). Find the probability a patient who tests positive actually has the disease $P(D | T)$.

1. Total Evidence $P(T) = P(T | D)P(D) + P(T | \neg D)P(\neg D) = (0.95)(0.01) + (0.05)(0.99) = 0.0095 + 0.0495 = \mathbf{0.0590}$.
2. $P(D | T) = \frac{P(T|D)P(D)}{P(T)} = \frac{0.0095}{0.0590} \approx \mathbf{0.161}$ (16.1%).

Topic 30: Probabilistic Reasoning & Bayesian Belief Networks

Full Joint Distribution Chain Rule for Bayesian Networks: A **Bayesian Network** is a Directed Acyclic Graph (DAG) where nodes represent random variables and directed edges represent direct conditional dependencies:

$$P(X_1, X_2, \dots, X_n) = \prod_{i=1}^n P(X_i | \text{Parents}(X_i))$$

Reduces storage from $O(2^n)$ in full joint table to $O(n \cdot 2^k)$ where k is maximum parent in-degree!

M4 Active Recall & Exam Questions

Q1. Define STRIPS. Explain the components of a planning action with a clear example. (8 Marks)

STRIPS (Stanford Research Institute Problem Solver) is a formal action representation language where world states are conjunctions of function-free positive literals.

An action is defined by: (1) *Name/Parameters*, (2) *Preconditions* (what must hold before execution), (3) *Add-List* (positive effects added to state), and (4) *Delete-List* (literals removed from state).

Example: `Stack(A, B)`

- PRE: `Clear(A), Clear(B), On(A, Table)`
- ADD: `On(A, B)`
- DEL: `Clear(B), On(A, Table)`

Module V: Machine Learning & Neural Networks — Complete 9-Topic Syllabus Tracker

Topic 31: What is Learning? (Experience to Performance Improvement)

Topic 33: Learning by Taking Advice (Expert Instruction Parsing)

Topic 35: Induction vs. Deduction in Machine Learning

Topic 37: Neural Net Learning (Perceptrons, Activations & Backpropagation)

Topic 39: Overfitting (High Variance, Noise Memorization & Regularization)

Topic 32: Rote Learning (Direct Memorization & Caching)

Topic 34: Learning from Examples (Inductive Supervised Classification)

Topic 36: Formal Learning Theory (Hypothesis Space, Bias & Variance)

Topic 38: Underfitting (High Bias, Low Capacity Models)

Topic 31 – 35: Forms of Machine Learning & Inductive Inference

Learning Paradigm	Operational Mechanism	Key Advantages & Tradeoffs
1. Rote Learning	Stores computed results and solutions directly in a lookup table / cache. Re-executing retrieves pre-computed solutions.	Instant $O(1)$ retrieval; zero generalization capability to unseen problem instances.
2. Learning by Advice	An external teacher/expert provides high-level instructions, which the system translates into internal operational rules.	Rapid knowledge bootstrapping; requires complex natural language understanding.
3. Learning from Examples	Induces general classification/prediction functions $y = f(x)$ from a set of labeled input-output pairs $\langle x_i, y_i \rangle$.	The foundation of modern machine learning; requires large representative datasets.
4. Induction vs. Deduction	• Induction: Derives general hypotheses from specific observations (probabilistic). • Deduction: Derives specific consequences from general axioms (guaranteed true).	Induction discovers new knowledge from data; Deduction proves theorems from axioms.

Topic 36: Formal Learning Theory (Hypothesis Space & Bias-Variance)

A learning algorithm searches through a **Hypothesis Space** $\mathcal{H}$ to select a candidate hypothesis $h \in \mathcal{H}$ that approximates the true underlying target function $f(x)$ with minimal generalization error on unseen data.

Concept	Mathematical / Theoretical Definition
Bias	Error from erroneous assumptions in the learning algorithm (e.g., assuming linear boundary for nonlinear data).
Variance	Error from sensitivity to small fluctuations in the training set (model memorizes random training noise).
Generalization	The ability of a trained model to accurately predict labels on novel, previously unseen test samples.

Topic 37: Neural Network Learning & Gradient Descent

1. Artificial Neuron (Perceptron) Formulation:

$$z = \sum_{i=1}^n w_i x_i + b = \mathbf{w}^T \mathbf{x} + b$$

$$y = f(z) \quad (\text{Activation Function})$$

- **Sigmoid:** $\sigma(z) = \frac{1}{1+e^{-z}} \in (0, 1)$ - **ReLU (Rectified Linear Unit):** $\text{ReLU}(z) = \max(0, z)$ - **Tanh:** $\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}} \in (-1, 1)$

2. Gradient Descent & Backpropagation Weight Update:

$$w_{ij} \leftarrow w_{ij} - \eta \frac{\partial \mathcal{L}}{\partial w_{ij}}$$

Where $\eta > 0$ is the learning rate and $\mathcal{L}$ is the Mean Squared Error loss function $\mathcal{L} = \frac{1}{2} \sum (t_k - y_k)^2$.

Topic 38 & 39: Underfitting vs. Overfitting

Characteristic	Underfitting (High Bias)	Overfitting (High Variance)
Root Cause	Model is too simplistic with insufficient expressive capacity.	Model is excessively complex with too many parameters.
Training Error	High (cannot fit training data).	Ultra-Low / Zero (memorizes training noise).
Testing / Validation Error	High (poor performance everywhere).	High (fails to generalize to unseen test data).
Engineering Fixes	• Increase network depth / model capacity. • Add more expressive polynomial features. • Reduce excessive regularization (λ).	• Gather more diverse training data. • Apply L_2 Weight Decay / L_1 Regularization. • Add Dropout layers & Early Stopping. • Prune decision trees / reduce parameters.

M5 Active Recall & Exam Questions

Q1. Differentiate between Underfitting and Overfitting. What techniques resolve overfitting in neural networks? (8 Marks)

- **Underfitting (High Bias):** Occurs when the model is too simple to capture underlying data trends, yielding high errors on both training and testing datasets.

- **Overfitting (High Variance):** Occurs when the model is overly complex, memorizing random noise and sample idiosyncrasies, leading to near-zero training error but high test error.

- **Techniques to Mitigate Overfitting:**

1. **Regularization (L_1/L_2):** Penalizes large weight magnitudes in the loss function ($\mathcal{L}_{\text{total}} = \mathcal{L} + \lambda \|\mathbf{w}\|^2$).
2. **Dropout:** Randomly deactivates a fraction p of hidden neurons during each training step.
3. **Early Stopping:** Halts training when validation loss stops improving.
4. **Data Augmentation:** Synthetically expands training set size through geometric transformations.